Materiały do sesji: kod, instrukcje i dema 001–010.

Osobista Fabryka Oprogramowania

Python, GitHub Copilot SDK i Inteligencja Białkowa

Tomasz Kopacz, Microsoft
tkopacz@microsoft.com

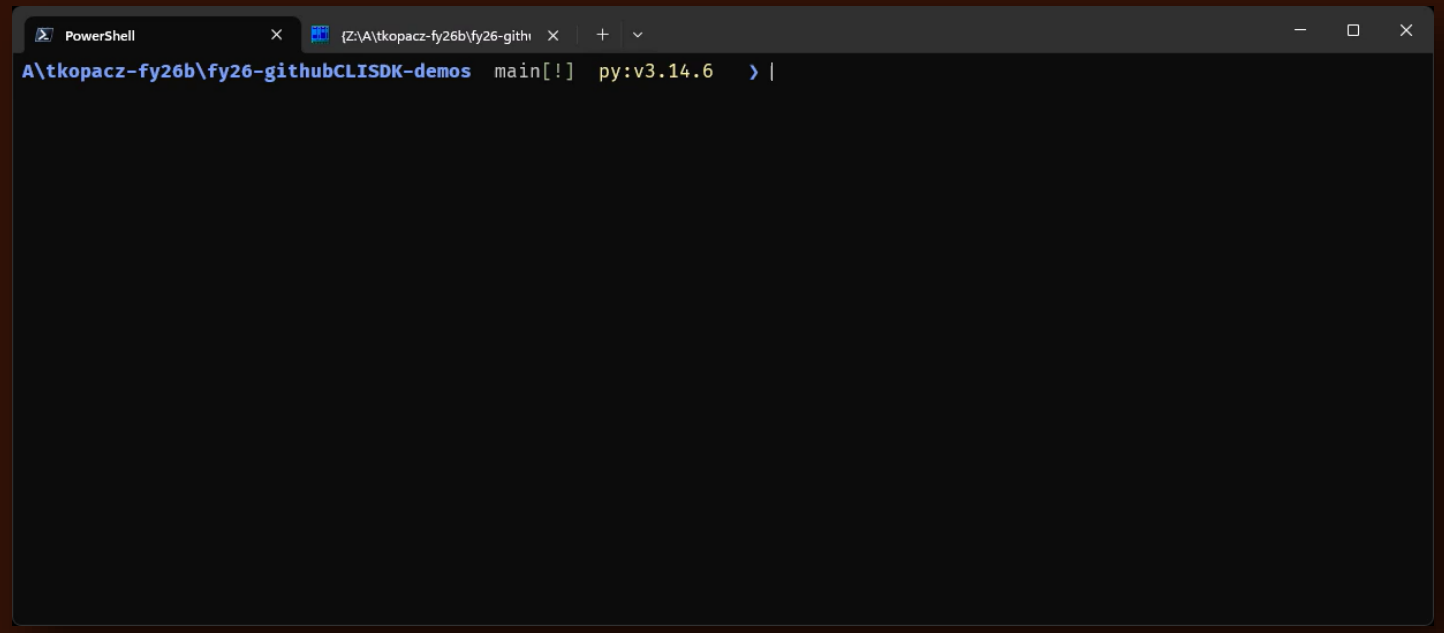
Kod, instrukcje i indeks dem 001–010



Od asystenta do fabryki

- Copilot w edytorze pomaga w pisaniu. Copilot CLI uruchamia agentów Copilot CLI SDK pozwala umieścić agenta we własnej aplikacji.
- Zamiast pisać pętlę agentową od zera, użyjemy tego samego runtime, który napędza Copilot CLI.
- Python będzie cienką warstwą integracji, a nie własnym frameworkiem agentowym.
- Efektem sesji będzie działający, wieloagentowy proces SDLC zbudowany głównie z możliwości SDK.

GitHub Copilot CLI – możliwości



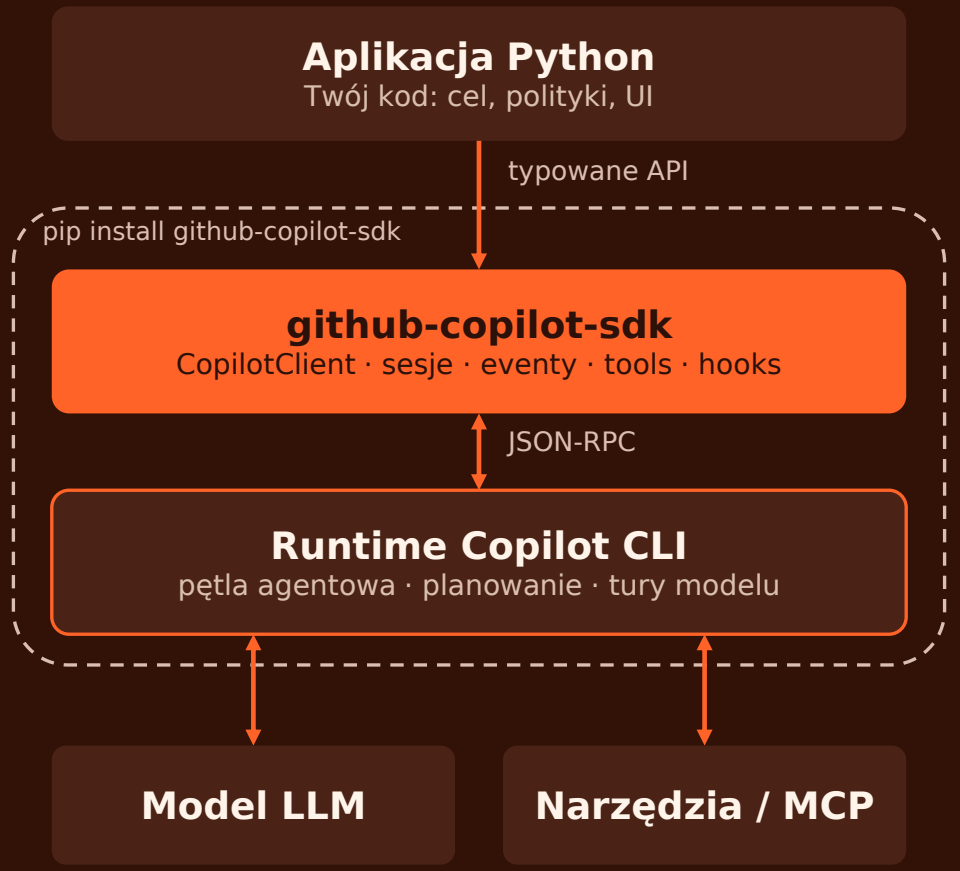

```
curl -fsSL https://gh.io/copilot-install | bash
winget install GitHub.Copilot
npm install -g @github/copilot
```

- Interaktywna pomoc
- Generowanie kodu i modyfikacja
- Wieloplikowa świadomość kontekstu
- Precyzyjna kontrola kontekstu
- Tryby: plan/auto/interactive/HYDRA
- Agenty, skille, ...
- Automatyzacja (też yolo)
- Narzędzi i polecenia powłoki
- Hooki zdarzeń cyklu życia
- Integracja z MCP
- Modele Claude , GPT, Gemini, ...

SDK to typowane API Pythona do runtime Copilot CLI, który sam prowadzi pętlę agentową.

Czym jest GitHub Copilot SDK

- Programistyczny interfejs do agenta GitHub Copilot CLI.
- SDK komunikuje się z runtime przez JSON-RPC i udostępnia typowane API dla Pythona.
- Copilot CLI wykonuje planowanie, wywołania narzędzi i kolejne tury modelu.
- SDK udostępnia sesje, eventy, narzędzia, uprawnienia, hooks, custom agents, MCP i telemetry.
- Python 3.11+, pakiet github-copilot-sdk; runtime jest dostarczany i wersjonowany razem z pakietem.



Demo 01 - Autoryzacja

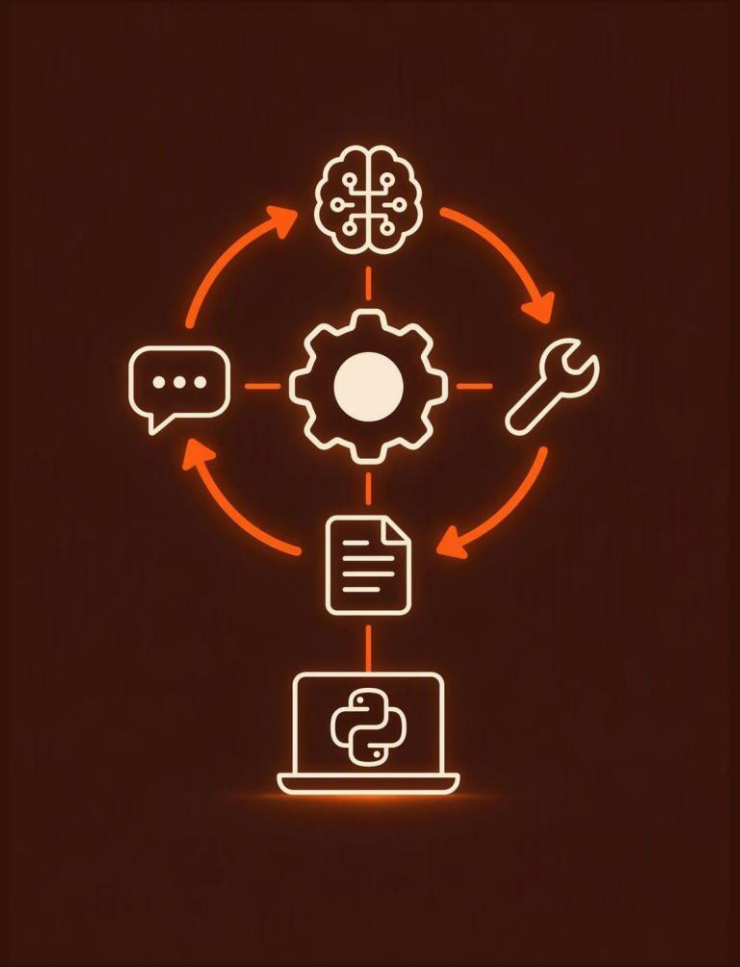
```python
async def run_request(
    self,
    *,
    model: str,
    prompt: str,
    use_logged_in_user: bool,
    provider: Mapping[str, Any] | None = None,
) -> None:
    self.output(f"Prompt: {prompt}")
    async with self.sdk.create_client(
        use_logged_in_user=use_logged_in_user
    ) as client:
        session_options: dict[str, Any] = {
            "model": model,
            # Auto-approval keeps this disposable demo moving; production hosts
            # should inspect each runtime permission request before deciding.
            "on_permission_request": self.sdk.approve_all_permissions(),
            # The SDK matches exact names and prevents matching MCP servers
            # from starting/authenticating in this session; this is not a
            # wildcard or a change to the CLI's global configuration.
            "disabled_mcp_servers": list(DISABLED_MCP_SERVERS),
            # The callback observes every emitted SDK event while deliberately
            # omitting payloads that could contain sensitive session data.
            "on_event": self.print_event_placeholder,
        }
        if provider is not None:
            # Supplying provider redirects this session's model calls to BYOK.
            session_options["provider"] = dict(provider)

        # A Copilot session owns conversation state and coordinates the runtime's
        # model turns, tool executions, permission requests, and event stream.
        async with await client.create_session(**session_options) as session:
            # The SDK still delivers events while waiting and returns after the
            # runtime emits session.idle, with the final assistant message if any.
            response = await session.send_and_wait(prompt)
```

SDK zdejmuje z nas pętlę agentową i infrastrukturę, zostawiając pełną kontrolę nad procesem.

Dlaczego SDK, a nie surowe API modelu

- Nie implementujemy pętli: model -> tool call -> wynik -> kolejna tura.
- Nie budujemy obsługi historii, strumieniowania, procesu CLI i protokołu komunikacji.
- Dostajemy gotowe narzędzia kodowe oraz mechanizm dodawania własnych narzędzi.
- Korzystamy z jednego API do modeli Copilot lub własnego dostawcy przez BYOK.
- Zachowujemy kontrolę nad instrukcjami, narzędziami, uprawnieniami i cyklem życia sesji. Ale dużo jest ich GOTOWYCH.
- Nasz kod koncentruje się na tym, co wyróżnia proces firmy.



Przykład analizowany:

beancount: https://github.com/beancount/beancount

A double-entry bookkeeping computer language that lets you define financial transaction records in a text file, read them in memory, generate a variety of reports from them, and provides a web interface.

Czyli:

1. Transakcje w pliku — np. wypłatę, zakupy, przelewy, raty kredytu.
2. Program odczytuje te zapisy i sprawdza ich poprawność.
3. Raporty — np. wydatki według kategorii, przychody, stan kont, zadłużenie czy wartość majątku.

Przykładowo: kupujesz jedzenie za 100 zł kartą:

konto bankowe: −100 zł,

wydatki na jedzenie: +100 zł.

Szukamy błędu w Beancount :

zapis z zerową ilością i podanym kosztem całkowitym (0 HOOL {{20 USD}})
powoduje DivisionByZero podczas przeliczania kosztu.
https://github.com/beancount/beancount/issues/1048

„Double-entry bookkeeping” oznacza księgowość podwójnego zapisu:

każda operacja jest zapisywana na co najmniej dwóch kontach,

tak aby wskazać, skąd środki pochodzą i dokąd trafiają.

Copilot SDK korzysta z tego samego harnessu co Copilot CLI – te wyniki dotyczą też naszych aplikacji.

Wynik: ta sama skuteczność za mniej — z wyborem modelu

Ta sama skuteczność

co Claude Code i Codex CLI na tym samym modelu

Niższy koszt zadania

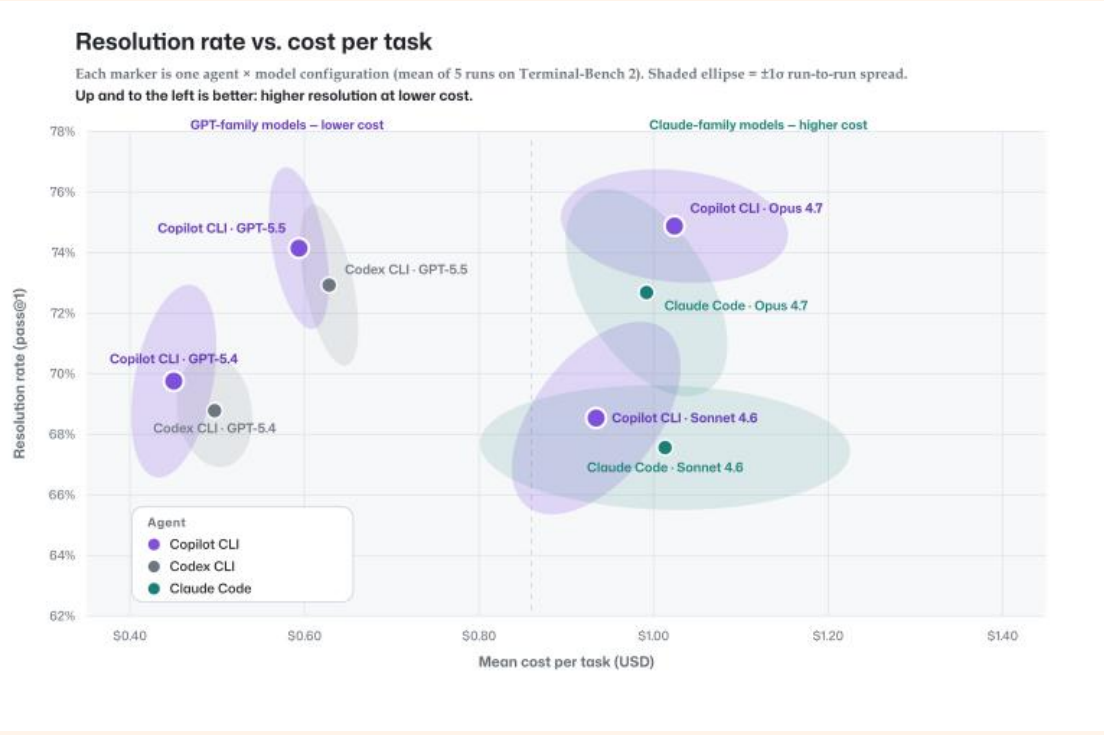
mniej tokenów w większości konfiguracji

Swoboda wyboru

tańszy GPT albo najmocniejszy Opus — w jednym narzędziu

To także harness aplikacji z Copilot SDK

SDK steruje tym samym runtime Copilot CLI: ta sama pętla agentowa, narzędzia i zarządzanie kontekstem – bez pisania własnego harnessu.



Na TerminalBench 2.0 Copilot nigdy nie jest mniej skuteczny ani droższy niż harness producenta z tym samym modelem.
Także: SWE-bench Verified i Pro, SkillsBench, TerminalBench 2.0, Win-Hill (wewnętrzny)

Źródło wykresu i danych: GitHub Blog, „Evaluating performance and efficiency of the GitHub Copilot agentic harness across models and tasks”, 25.06.2026 (TerminalBench 2.0).

Subskrybujemy eventy sesji zamiast czekać na odpowiedź – UI reaguje na tokeny, narzędzia i koniec pracy agenta.

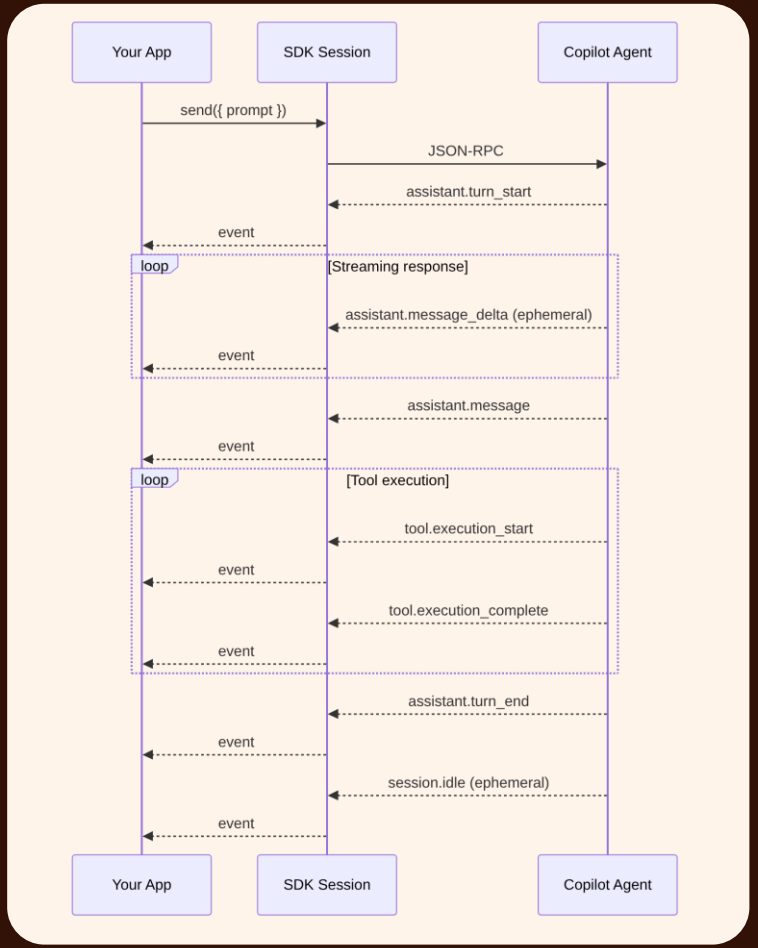
Model zdarzeń — nie blokuj wątku, reaguj na eventy

```python
from copilot.session_events import SessionEventType as E
done = asyncio.Event()

def on_event(event):
    match event.type:
        case E.ASSISTANT_MESSAGE_DELTA:   # wymaga streaming=True
            print(event.data.delta_content, end="", flush=True)
        case E.TOOL_EXECUTION_START:
            show_spinner(event.data.tool_name)
        case E.SESSION_IDLE:
            done.set()

session.on(on_event)         # subskrybuj przed send()
await session.send(prompt)   # nie blokuje — eventy płyną w tle
await done.wait()
```

`assistant.message_delta`	Token strumieniowy – wyświetl na żywo
`assistant.message`	Pełna odpowiedź (może zawierać toolRequests)
`tool.execution_start`	Agent woła narzędzie – pokaż spinner
`tool.execution_complete`	Wynik narzędzia: success / error
`permission.requested`	Agent prosi o zgodę na operację
`session.idle`	Koniec pracy – sygnał dla SSE / kolejki
`session.error`	Błąd sesji, np. errorType = rate_limit



Kilka linii Pythona wystarczy, by uruchomić klienta, sesję i pierwsze zapytanie do agenta.

Start w Pythonie

```
pip install github-copilot-sdk

from copilot import CopilotClient
from copilot.session import PermissionHandler

# Login CLI; klient uruchamia runtime i zamyka go przy wyjściu.
async with CopilotClient() as client:
    # Sesja ma historię; auto wybiera model, approve_all upraszcza demo.
    async with await client.create_session(
        model="auto",
        on_permission_request=PermissionHandler.approve_all,
    ) as session:
        # Czekaj na session.idle; otrzymasz ostatnią odpowiedź asystenta.
        response = await session.send_and_wait("Przeanalizuj wymaganie w issue")
        print(response.data.content)
```

approve_all upraszcza demo; nie jest domyślną ani ZALECANĄ polityką produkcyjną.

CopilotClient, lifecycle runtime i autoryzacja

- CopilotClient() domyślnie uruchamia zgodny runtime dostarczony z SDK.
- Context manager automatycznie wykonuje start i cleanup.
- Możliwe są transporty stdio, TCP, zewnętrzny serwer oraz eksperymentalny in-process FFI.
- Zewnętrzny Copilot CLI server przydaje się przy debugowaniu i współdzieleniu zasobów.
- working_directory określa repozytorium widziane przez agenta.

Autoryzacja – domyślnie konto GitHub

- **Login w Copilot CLI:** copilot → /login (OAuth device flow), token w keychainie; CopilotClient() używa go automatycznie.
- **CI/CD i serwery:** token w zmiennej środowiskowej COPILOT_GITHUB_TOKEN / GH_TOKEN / GITHUB_TOKEN.
- **Aplikacje w imieniu użytkownika:** GitHub OAuth App – wymagana subskrypcja Copilot.

Własny klucz do modeli (BYOK)

- **1. Klucz od dostawcy** (Foundry / Azure OpenAI, OpenAI, Anthropic, Ollama) w zmiennej środowiskowej, np. FOUNDRY_API_KEY.
- **2. create_session(model=..., provider={...})** z polami type, base_url, api_key.
- **3. type:** "openai" (też OpenAI-compatible), "azure" lub "anthropic"; wire_api: "responses" lub "completions".
- **Efekt:** BYOK omija logowanie GitHub – rozliczenie bezpośrednio u dostawcy.

Sesja jest jednostką pracy

- create_session() ustala model, katalog roboczy, narzędzia i politykę uprawnień.
- Sesja przechowuje rozmowę, wyniki narzędzi i kontekst planowania.
- send() rozpoczyna pracę bez blokowania; send_and_wait() czeka na session.idle.
- disconnect() zwalnia zasoby, ale może pozostawić stan do wznowienia.
- Oddzielna sesja na zadanie ułatwia izolację, audyt i kontrolę kosztu.

```
# Lokalnie: metadane sesji z dysku → pełna historia eventów wybranych sesji
sessions = await client.list_sessions(SessionListFilter(working_directory=os.getcwd()))
for meta in sorted(sessions, key=lambda m: m.modified_time, reverse=True):
    if "beancount" not in (meta.summary or "").lower():   # szybki filtr po summary
        continue
    s = await client.resume_session(meta.session_id,
                                    on_permission_request=PermissionHandler.approve_all)
    for ev in await s.get_events():                        # cała historia sesji
        match ev.data:
            case AssistantMessageData(content=c) if "pytest" in c:
                print(meta.session_id, meta.modified_time, c[:80])
    await s.disconnect()                                    # stan zostaje na dysku
```

Sesje leżą lokalnie w ~/.copilot/session-state/<id>/ · filtry: working_directory, git_root, repository, branch
Też polecenie **/chronicle search** <tu szukany text>

Demo 02 - Sesja

```
148
149                 # Intermediate events still reach on_event while this waits. After
150                 # session.idle, the returned event should hold the final assistant
151                 # message produced by the completed model-and-tool loop.
152                 response = await session.send_and_wait(
153                     configuration.prompt,
154                     timeout=configuration.timeout_seconds,
155                 )
156                 if response is None or not isinstance(
157                     response.data, AssistantMessageData
158                 ):
159                     raise RuntimeError(
160                         "Copilot reached session.idle without a final assistant message."
161                     )
162                 self.output("[lifecycle] Session idle")
163                 self.output("\nCopilot response:")
164                 self.output(response.data.content)
165
166             self.output("[lifecycle] Session closed")
167
168         self.output("[lifecycle] Copilot client closed")
169
170     def run(self, configuration: DemoConfiguration) -> int:
171         try:
172             asyncio.run(self.execute(configuration))
173         except ConfigurationError as error:
```

roblems Output Test Results Terminal Ports Debug Console

```
PS Z:\A\tkopacz-fy27-pub\fy27-gh-sdk-python-sdlc>  & 'z:\A\tkopacz-fy27-pub\fy27-gh-sdk-python-sdlc\.venv
ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53058' '--' 'Z:\A\tkopacz-fy27-pub\f
[event] pending_messages.modified
[event] session.title_changed
[event] session.auto_mode_resolved
[event] session.skills_loaded
[event] session.mcp_servers_loaded
[event] session.tools_updated
[event] user.message
[event] system.message
```

Agent dostaje wbudowane narzędzia Copilot CLI i nasze własne, a my decydujemy, co widzi i czego może użyć.

Narzędzia: wbudowane, własne i uprawnienia

Wbudowane: pliki, grep, shell · **Własne:** @define_tool + Pydantic · **Kontrola:** allowlista, blocklista, uprawnienia

1 · Definicja i widoczność narzędzia

```
class IssueParams(BaseModel):
    number: int = Field(description="Numer issue")

@define_tool(description="Pobiera zatwierdzone wymaganie")
async def get_requirement(params: IssueParams) -> str:
    return await tracker.get_issue(params.number)

# Rejestracja = widoczność w liście narzędzi modelu
session = await client.create_session(
    tools=[get_requirement],            # własny tool
    available_tools=["view", "grep",
                     "get_requirement"], # allowlista
    excluded_tools=["shell"],           # blocklista
    on_permission_request=on_permission)

# Model widzi tylko nazwę, opis i schemat JSON:
#   get_requirement(number: integer)
```

2 · Wywołanie i uprawnienia

```
await session.send_and_wait(
    {"prompt": "Zaimplementuj wymaganie z issue #42"})

# Model sam wybiera tool → event tool.execution_start
#   get_requirement({"number": 42})
# SDK uruchamia handler, wynik wraca do pętli agenta

# Uprawnienia: decyzja przed każdym użyciem narzędzia
def on_permission(req, invocation):
    if req.kind == "read":
        return {"kind": "approved"}        # czytanie: OK
    if req.kind in ("shell", "write"):
        return {"kind": "denied-by-rules"}  # zakaz
    return {"kind": "approved"}
```

Fragment poglądowy; pełny, uruchamialny przykład znajdziesz w indeksie dem.

Demo 03 – Custom Tool, get_requirement

```python
def build_requirement_tool(fixture_path: Path) -> Tool: fixture_path = WindowsPa
    # Copilot publishes this JSON schema to the model. defer="never" keeps the
    # tool in the first turn instead of hiding it behind tool search.
    # skip_permission lets this read-only lookup run without a permission
    # prompt; do not copy that flag onto tools that change files or call
    # external systems.
    @define_tool(
        description=(
            "Pobiera zatwierdzone wymaganie i kryteria akceptacji z lokalnego "
            "snapshotu issue beancount."
        ),
        defer="never",
        skip_permission=True,
    )
    def get_requirement(params: IssueParams) -> str:
        return RequirementCatalog(fixture_path).get(params.number)

    return get_requirement
```

Kryteria akceptacji wyciągnięte ze zgłoszonego issue

Demo 04 – Security + Custom Tool

```python
async with await client.create_session(
    model=configuration.model,
    working_directory=str(policy.working_directory),
    tools=[read_tool],
    available_tools=[ProjectReadGuard.TOOL_NAME],
    # The SDK blocks these exact names for this session while
    # preserving the custom reader and its independent path guard.
    disabled_mcp_servers=list(DISABLED_MCP_SERVERS),
    # on_permission_request - but - we are not using it in this demo.
    hooks={"on_pre_tool_use": guard.check},
    enable_file_hooks=False,
    on_event=self.show_event,
) as session:
```

```javascript
const READ_ONLY_TOOLS = ["read_file", "glob", "grep", "view"];

const session = await client.createSession({
  hooks: {
    onPreToolUse: async (input) => {
      if (!READ_ONLY_TOOLS.includes(input.toolName)) {
        return {
          permissionDecision: "deny",
          permissionDecisionReason: `Only read-only tools are allowed. "${input.toolName}" was blocked.`,
        };
      }
      return { permissionDecision: "allow" };
    },
  },
  onPermissionRequest: async () => ({ kind: "approve-once" }),
});
```

Hooks pozwalają walidować, modyfikować i audytować pracę agenta na każdym etapie sesji.

Hooks: kontrola bez pisania własnej pętli

`on_pre_tool_use` zezwól / odrzuć / zmień argumenty

`on_post_tool_use` dodaj kontekst po wyniku

`on_post_tool_use_failure` wskazówki naprawcze

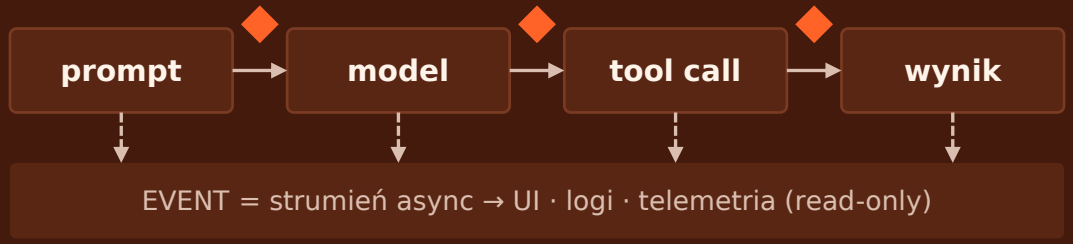
`on_user_prompt_submitted` waliduj / uzupełnij

`on_session_start / _end` init, audyt, cleanup

`on_error_occurred` retry, skip albo abort

Kiedy hook, a kiedy event?

◆ HOOK = synchroniczna bramka – agent czeka na decyzję

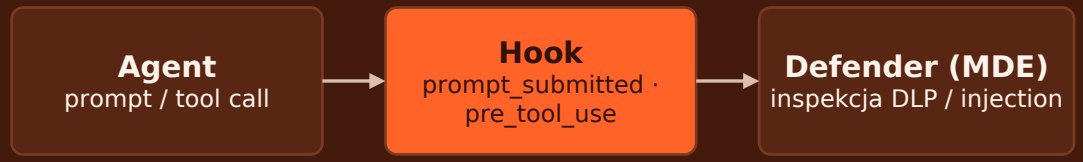


Hook, gdy musisz wpłynąć na agenta: zablokować, zmienić argumenty, dodać kontekst, wymusić politykę.

Event, gdy tylko obserwujesz: streaming tokenów, spinner, logi, metryki – bez wpływu na przebieg.

Przykład: Defender for Endpoint jako hook

AI agent runtime protection (preview) – np. GitHub Copilot CLI



✓ **allow** treść trafia do modelu w chmurze

× **block** sekrety, dane wrażliwe (np. .env) lub ukryta instrukcja wysyłki na zewnątrz → akcja zatrzymana przed wykonaniem + alert w Defender XDR

Źródło: learn.microsoft.com – AI agent runtime protection with MDE (preview)

Serwery MCP podłączają GitHub, bazy danych i wewnętrzne API bez pisania własnych wrapperów.

MCP: gotowe integracje zamiast wrapperów

- SDK obsługuje lokalne serwery MCP przez stdio oraz zdalne przez HTTP/SSE.
- MCP może udostępnić GitHub, tracker zadań, bazę danych lub wewnętrzne API.
- Lista tools ogranicza funkcje widoczne dla agenta.
- Serwer MCP może być dostępny dla całej sesji albo tylko dla wybranego custom agenta.
- Korzystamy ze standardowego protokołu zamiast pisać osobną integrację dla każdego systemu.

Przykładowe serwery MCP i ich zadania dla agenta

MICROSOFT / GITHUB

GitHub

- Repozytoria, issues i pull requesty
- Wyszukiwanie kodu i historii zmian
- Status GitHub Actions, review PR

MICROSOFT

Azure DevOps

- Work items i boardy w sprincie
- Repozytoria, PR-y i pipeline'y
- Wiki projektu i test plans

MICROSOFT 365

Work IQ

- Kontekst pracy z Microsoft 365
- Maile, spotkania, czaty Teams
- Dokumenty i ustalenia zespołu

ATLASSIAN

Jira

- Wyszukiwanie zgłoszeń przez JQL
- Tworzenie i aktualizacja tasków
- Zmiana statusów, link do Confluence

GITLAB

GitLab

- Projekty, issues i merge requesty
- Pipeline'y CI/CD i ich wyniki
- Komentarze i przegląd zmian

Serwery MCP włączamy w konfiguracji sesji – agent widzi ich funkcje jak zwykłe tools.

Jak włączyć MCP dla własnego agenta

```
# Serwery MCP przekazujemy przy tworzeniu sesji — runtime sam je uruchamia.
async with await client.create_session(
    model="auto",
    on_permission_request=PermissionHandler.approve_all,
    mcp_servers={
        "github": {  # zdalny, HTTP
            "type": "http", "url": "https://api.githubcopilot.com/mcp/",
            "headers": {"Authorization": f"Bearer {GH_TOKEN}"},
            "tools": ["get_issue", "list_pull_requests"],  # tylko te
        },
        "azure-devops": {  # lokalny, stdio
            "type": "local", "command": "npx",
            "args": ["-y", "@azure-devops/mcp", "contoso"], "tools": ["*"],
        },
    },
) as session:
    await session.send_and_wait("Pobierz issue #42 i powiązany work item z ADO")
```

Work IQ: "type": "local", "command": "npx", "args": ["-y", "@microsoft/workiq", "mcp"] · Jira, GitLab: "type": "http" + URL serwera MCP
Tylko dla jednego agenta: ten sam słownik mcp_servers w definicji custom_agents=[{...}] · approve_all wyłącznie na potrzeby dema.

Każda rola SDLC to custom agent z dobranymi narzędziami – delegację wykonuje runtime, a aplikacja pilnuje polityk.

Fabryka SDLC jako konfiguracja SDK

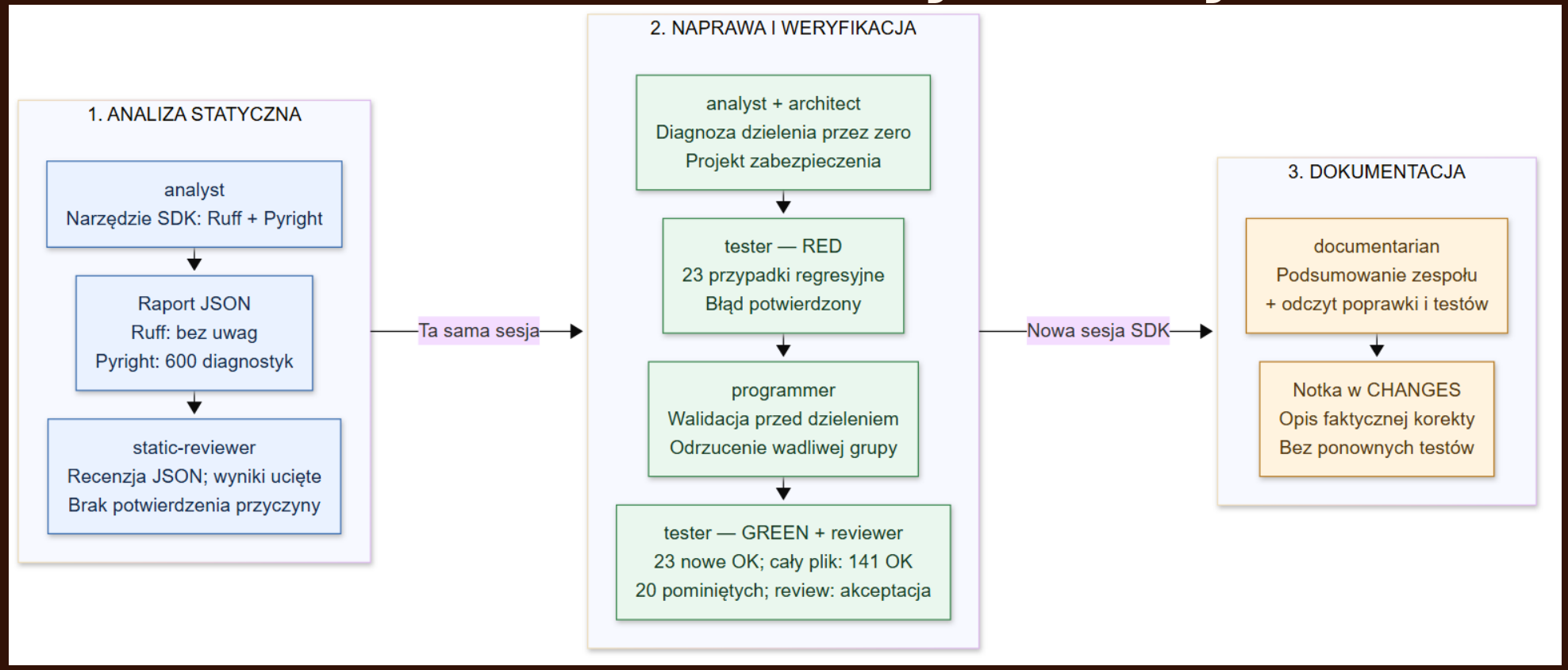


Nie piszemy sześciu osobnych pętli wywołań modelu.

Definiujemy specjalistów, ich granice i narzędzia.

Przewidywalne etapy: jawny wybór agenta. Klasyczny workflow

Demo (nooo...) 05 – Fabryka. Przykład uruchomienia:




```
Copilot response:
Zmieniono wyłącznie `CHANGES`: dodano krótką notkę o #1048 — zerowa liczba jednostek z kosztem całkowitym zgłasza teraz `Amount
iast nieobsłużonego wyjątku; obliczenia dla niezerowych jednostek pozostają bez zmian.

Przejrzano poprawkę i regresje w obu wskazanych plikach. Nie uruchamiano testów ani nie wykonywano commit, push lub rollbacku.
[rollback] Disabled: workspace changes kept for inspection. No commit or push.
[lifecycle] Copilot client closed
[usage] AIC=49.6904499; status=reported; reported calls=52; calls without billing=0. SDK-reported usage, not a billing invoice.
(.venv) PS Z:\A\tkopacz-fy27-pub\fy27-gh-sdk-python-sdlc>
```

Demo 06 – Decyzja człowieka w sesji

```python
def elicit(self, context: ElicitationContext) -> ElicitationResult:
    # ask_user_variant="elicitation" routes the model's question to this
    # callback; the selected answer returns directly to the same agent loop.
    #
    # Elicitation (wywołanie) asks, “Which behavior do you choose?”,
    # while permission asks, “Is this operation allowed?”
    self.output("[elicitation] zero-units semantics:")
    for key, description in CHOICES.items():
        self.output(f"  {key}: {description}")
    answer = self.ask("Choose reject, zero_cost or keep_raising: ").strip()
    if answer not in CHOICES:
        self.output("[elicitation] invalid choice; request declined")
        return {"action": "decline"}
    schema = context.get("requestedSchema", {})
    properties = schema.get("properties", {})
    if not properties:
        self.output("[elicitation] missing form fields; request declined")
        return {"action": "decline"}
    field = next(iter(properties))
    return {"action": "accept", "content": {field: answer}}
```

Oczywiście można używać MCP, pracować z historią rozmowy i restartować sesje!

Demo 08 – GitHub MCP (opcje:historia i restart)

```
PREPARE_PROMPT = (
    "Read beancount/beancount issue #1048 AND its CHANGES with GitHub MCP, "
    "then act as documentalist: call "
    "documentalist_update_versions once with this exact single-line note: "
    "'Proposed documentation for issue #1048: explain DivisionByZero with "
    "0 HOOL {{20.00 USD}} and the expected booking error.' "
    "The tool adds it at the top of the fork's local "
    "CHANGES. Do not claim the bug is fixed. Do not commit, push or "
    "create a PR; leave the changed file for human review."
)
```

NIE MA IDEALNEGO MODELU – zależy od problemu,

Demo 08 – Trzy (cztery) modele, jeden harness

```
[response] gpt-6.1-sol:
1. **Zerowa liczba jednostek — obie równoważne składnie**
   - **Działanie:** Wywołać `load_file` dla dwóch wariantów reprodukcji: `0 HOOL {{20.00 USD}}` oraz `0 HOOL {0 # 20.00 USD}`.
   - **Oczekiwany wynik:** Każde wywołanie zwraca `(entries, errors, options_map)` bez wyjątku. Lista `errors` zawiera błąd księgowania z komunikatem, że kosztu całkowitego nie można użyć przy zerowej liczbie jednostek.
   - **Uzasadnienie:** Kryteria wymagają jednakowego odrzucenia obu zapisów, jednoznacznego komunikatu i braku `decimal.DivisionByZero` wydostającego się z loadera.

2. **Zerowe jednostki i zerowy koszt całkowity**
   - **Działanie:** Sprawdzić `compute_cost_number` dla zerowej liczby jednostek i kosztu całkowitego `0.00 USD`; następnie wczytać wpisy `0 HOOL {{0.00 USD}}` oraz `0 HOOL {0 # 0.00 USD}`.
   - **Oczekiwany wynik:** Funkcja zwraca `None`, nie wykonując dzielenia. Loader zwraca błąd o niedopuszczalnym koszcie całkowitym przy zerowych jednostkach, bez wyjątku dziesiętnego.
   - **Uzasadnienie:** Warunek ochronny dotyczy zerowej liczby jednostek, niezależnie od wartości kosztu. Ten przypadek wykrywa poprawkę chroniącą jedynie przed `20 / 0`, ale przepuszczającą `0 / 0`.

3. **Niezerowe jednostki — zachowanie poprawnych obliczeń**
   - **Działanie:** Wczytać poprawnie zbilansowane transakcje z `2 HOOL {{20.00 USD}}` oraz `-2 HOOL {{20.00 USD}}`, porównując wynik z dotychczasowym zachowaniem.
   - **Oczekiwany wynik:** Koszt jednostkowy pozostaje odpowiednio `10.00 USD` i `-10.00 USD`; nowa kontrola zera nie zgłasza dla tych wpisów błędu.
   - **Uzasadnienie:** Kryteria wymagają zachowania dotychczasowych obliczeń dla niezerowych jednostek. Wariant ujemny wykrywa zbyt szeroki warunek, np. `units_number <= 0` zamiast sprawdzenia równości zeru.
```

```
[response] gpt-6-astra:
1. **Zerowa liczba jednostek, koszt całkowity `{{20.00 USD}}`**
   - **Działanie:** Wywołać `load_file` dla reprodukcji z fixture: `0 HOOL {{20.00 USD}}` oraz przeciwstawny wpis `-20.00 USD`.
   - **Oczekiwany wynik:** Loader zwraca `(entries, errors, options_map)` bez wyjątku. Lista `errors` zawiera błąd księgowania z komunikatem, że kosztu całkowitego nie można używać przy zerowej liczbie jednostek.
   - **Uzasadnienie:** Kryteria wymagają błędu w zwracanej liście zamiast wyjątku oraz określają sens komunikatu. Sam brak `decimal.DivisionByZero` nie wystarcza.

2. **Alternatywny zapis kosztu przy zerowych jednostkach**
   - **Działanie:** Powtórzyć test loadera z `0 HOOL {0 # 20.00 USD}`. Dodatkowo przetestować bezpośrednio `compute_cost_number` dla zerowej liczby jednostek i specyfikacji kosztu odpowiadających obu zapisom.
   - **Oczekiwany wynik:** Loader odrzuca alternatywny zapis tak samo jak `{{20.00 USD}}`, z błędem o koszcie całkowitym przy zerowych jednostkach, bez wyjątku. `compute_cost_number` zwraca `None`.
   - **Uzasadnienie:** Kryteria osobno wymagają równoważnej obsługi `{0 # 20.00 USD}` oraz zwracania `None` przez funkcję pomocniczą. Sprawdza to, czy poprawka nie ogranicza się do jednego zapisu lub samego przechwyceni
a wyjątku w loaderze.

3. **Niezerowa liczba jednostek — kontrola regresji**
   - **Działanie:** W reprodukcji zastąpić `0 HOOL` przez `2 HOOL`, zachowując koszt całkowity `20.00 USD` i przeciwstawny wpis `-20.00 USD`. Sprawdzić warianty `{{20.00 USD}}` i `{0 # 20.00 USD}`.
   - **Oczekiwany wynik:** Loader nie zgłasza błędu dla poprawnej transakcji; koszt jednostkowy wynosi dokładnie `10.00 USD`.
   - **Uzasadnienie:** Kryteria wymagają zachowania dotychczasowego przeliczania kosztu dla niezerowych jednostek. Test wykrywa zbyt szeroką blokadę, która odrzucałaby wszystkie koszty całkowite.
```

```
[response] claude-opus-5.5:
Oto trzy przypadki brzegowe. Każdy wynika z kryteriów akceptacji w `beancount-1048.json`:

**1. Zapis `{0 # 20.00 USD}`, czyli koszt łączny zapisany ze znakiem `#`**
- **Działanie:** wywołaj `load_file` na księdze z repro, w której zamiast `0 HOOL {{20.00 USD}}` jest `0 HOOL {0 # 20.00 USD}`.
- **Oczekiwany wynik:** brak wyjątku i dokładnie jeden wpis w `errors`. Komunikat mówi, że kosztu łącznego nie można użyć przy zerowej liczbie jednostek.
- **Uzasadnienie:** ten zapis ma zostać odrzucony tak samo jak `{{...}}` (kryteria 2 i 3), a żadna forma specyfikacji kosztu nie może wypuścić `DivisionByZero` (kryterium 6).

**2. Zero ze znakiem minus i ze skalą: `-0.00 HOOL {{20.00 USD}}`**
- **Działanie:** wywołaj `load_file` na takim wpisie. Dodatkowo wywołaj bezpośrednio `compute_cost_number` z liczbą jednostek `Decimal("-0.00")`.
- **Oczekiwany wynik:** `load_file` zwraca ten sam błąd księgowania, a `compute_cost_number` zwraca `None`. Żadne wywołanie nie rzuca wyjątku.
- **Uzasadnienie:** `Decimal("-0.00")` jest równe zeru, ale ma inną postać niż `0`. Poprawka musi sprawdzać wartość, a nie tekst ani tożsamość obiektu (kryteria 1, 4 i 6).

**3. Najmniejsza niezerowa liczba jednostek: `0.0001 HOOL {{20.00 USD}}`**
- **Działanie:** wywołaj `load_file` na wpisie `0.0001 HOOL {{20.00 USD}}` z kontrą `-20.00 USD`.
- **Oczekiwany wynik:** lista `errors` jest pusta, a koszt jednostkowy wynosi `200000 USD`, tak jak przed poprawką.
- **Uzasadnienie:** test sprawdza, że zabezpieczenie zatrzymuje tylko zero. Bardzo małe niezerowe ilości nadal mają dawać ten sam koszt jednostkowy (kryterium 5).
```

HydraFusion | Ensemble | „Auto v2”

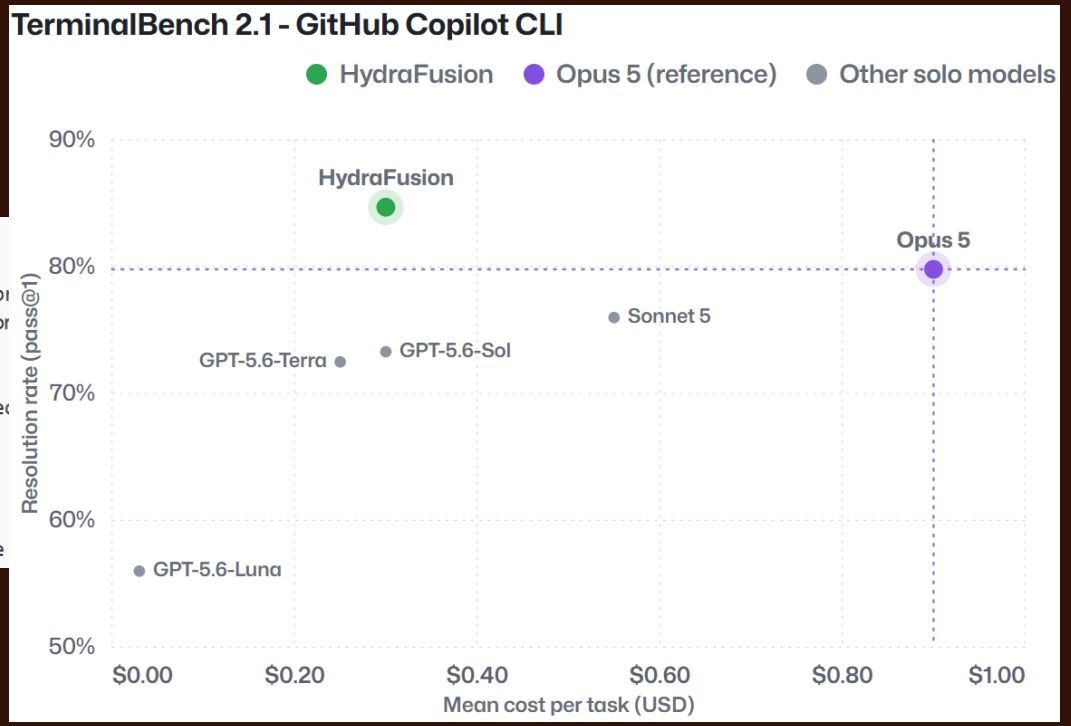
- Model = Auto – wybór do DANEGO zadania
- https://github.blog/ai-and-ml/github-copilot/project-hydrafusion-frontier-quality-via-multi-model-orchestration/

Też: https://arxiv.org/abs/2605.17106

```
[response] requested=auto; actual model=gpt-6-luna:
1. **Działanie:** Załaduj transakcję z `0 HOOL {{20.00 USD}}` i przeciwnym postingiem `-20.00 USD`.
   **Oczekiwany wynik:** `load_file` nie zgłasza wyjątku; zwraca błąd informujący, że koszt całkowity nie może być użyty pr
   **Uzasadnienie:** Bezpośrednio sprawdza kryteria dotyczące zapisu z usterki, wpisu błędu zamiast wyjątku oraz treści kor

2. **Działanie:** Powtórz przypadek z zapisem kosztu `{0 # 20.00 USD}` zamiast `{{20.00 USD}}`.
   **Oczekiwany wynik:** Loader nie zgłasza wyjątku i zwraca analogiczny błąd o koszcie całkowitym przy zerowej liczbie jed
   **Uzasadnienie:** Kryteria wymagają odrzucenia tej alternatywnej pisowni w taki sam sposób.

3. **Działanie:** Załaduj kontrolny posting `2 HOOL {{20.00 USD}}`.
   **Oczekiwany wynik:** Brak błędu; koszt jednostkowy nadal wynosi `10.00 USD`.
   **Uzasadnienie:** Sprawdza, że poprawka nie zmienia dotychczasowego obliczenia dla niezerowej liczby jednostek, zgodnie
```



ACP = Agent Client Protocol,

Demo 09 – ACP + Web



- Standard przemysłowy – Github, JetBrains, ZED, OpenCode, Claude Code …
  Komunikacja pomiędzy „edytorem kodu” a „agentem kodującym”
  JSON-RPC 2.0

  Ręcznie start :
  copilot --acp --port 3000 --effort=max --available-tools="bash,view"

- https://agentclientprotocol.com/get-started/introduction

  https://github.com/agentclientprotocol/

  https://docs.github.com/en/copilot/reference/copilot-cli-reference/acp-server

Copilot CLI = wygodny terminal który można ROZBUDOWYWAĆ (zamiast pisać od nowa). A harness systemowy można zastąpić (tylko trzeba pisać w JavaScript)

Demo 10 – Rozszerzenie CLI

- /demo | /demo stats | /demo joke

```
async run(model) {
    this.cancelled = false;
    this.phase = "configuration";
    this.client = this.createClient();
    try {
        this.phase = "startup";
        await this.client.start();
        this.checkCancelled();
        await this.log(`JOKE_START model=${model} mode=empty system=replace`);
        const czech = await this.stage(model, "cs", JOKE_SYSTEM, "Tell one joke.");
        await this.log(`JOKE_CS ${czech}`);
```

Najważniejsze wnioski

- GitHub Copilot SDK daje programistyczny dostęp do kompletnego runtime agentowego.
- W Pythonie konfigurujemy sesje, tools, agentów i polityki zamiast pisać pętlę od zera.
- Custom agents i MCP pozwalają zbudować zespół dopasowany do istniejącego SDLC.
- Eventy, hooks, permissions, persistence i telemetry dają punkty kontroli potrzebne w realnym systemie.
- Inteligencja białkowa pozostaje właścicielem celu, wyjątków i decyzji o ryzyku.

Podsumowanie sesji i repozytorium z kodem oraz demami.

Stawiam i sprawdzam realizacje celu

Runtime robi pętlę agentową.
Człowiek odpowiada za cel, jakość i ryzyko. Ale – deterministycznie i LLM-em

Kod, instrukcje i indeks dem 001–010



Dema 001–005 prowadzą od uwierzytelniania po zespół SDLC z analizą statyczną.

Indeks dem 001–005

001 **Uwierzytelnianie i BYOK** README ↗
demos/001-auth

002 **Pierwsza pełna sesja** README ↗
demos/002-initialsession

003 **Wymaganie przez własny tool** README ↗
demos/003-custom-tool

004 **Granice odczytu i pre-tool hook** README ↗
demos/004-custom-tool-secure

005 **Zespół SDLC i analiza statyczna** README ↗
demos/005-sdlc-team

Dema 006–010 pokazują człowieka w pętli, MCP, wybór modelu, ACP i rozszerzenie CLI.

Indeks dem 006–010

006 **Decyzja człowieka w sesji** README ↗
demos/006-human-policy

007 **GitHub MCP, historia i restart** README ↗
demos/007-mcp-resume

008 **Wiele modeli, jeden harness** README ↗
demos/008-model-choice

009 **Własny interfejs WWW przez ACP** README ↗
demos/009-acp-web

010 **Natywne rozszerzenie CLI** README ↗
demos/010-cli-extension

Linki do dokumentacji SDK, przewodników oraz repozytorium z kodem i demami tej sesji.

Materiały źródłowe

GitHub Copilot SDK ↗

https://github.com/github/copilot-sdk

Python SDK reference ↗

https://github.com/github/copilot-sdk/blob/main/python/README.md

Getting started ↗

https://github.com/github/copilot-sdk/blob/main/docs/getting-started.md

SDK feature guides ↗

https://github.com/github/copilot-sdk/blob/main/docs/features/README.md

Repozytorium tej sesji ↗

https://github.com/tkopacz/fy27-gh-sdk-python-sdlc-public

Indeks dem i instrukcje przygotowania ↗

https://github.com/tkopacz/fy27-gh-sdk-python-sdlc-public/blob/main/README.md